Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ
DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▸ Beauregard
Nom d'usage ▸ Beauregard
Prénom ▸ Isabelle
Adresse ▸ 70 rue du pont, 75000 Paris

Titre professionnel visé

Développeur Web et Web Mobile | DWWM

MODALITE D'ACCES :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

http://travail-emploi.gouv.fr/titres-professionnels

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile sécurisée	**p.**	**5**
▸ Arcadia ..... p.	p.	5
▸ Mini jeu parallaxe JS ..... p.	p.	8
▸ Intitulé de l'exemple n° 3 ..... p	p.	
Développer la partie back-end d'une application web ou web mobile sécurisée	**p.**	
▸ Arcadia ..... p.	p.	10
▸ QuizApp ..... p.	p.	14
▸ Intitulé de l'exemple n° 3 ..... p	p.	
Titres, diplômes, CQP, attestations de formation *(facultatif)*	**p.**	
Déclaration sur l'honneur	**p.**	18
Documents illustrant la pratique professionnelle *(facultatif)*	**p.**	
Annexes *(Si le RC le prévoit)*	**p.**	20

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ▸ ***Zoo Arcadia***

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Ce projet a été réalisé dans le cadre de l'Évaluation en Cours de Formation (ECF) de ma formation DWWM. Le sujet m'a été donné par les formateurs de STUDI.

Installation et configuration de mon environnement de travail :
J'ai commencé par créer un nouveau repository (stockage centralisé et distant) sur mon compte GitHub en cliquant sur **"New"**, que j'ai nommé **laravel-project-arcadia** et défini en mode public avant de cliquer sur **"Create repository"**.
Ensuite, dans mon IDE (Integrated Development Environment) **PHPStorm**, j'ai utilisé l'option **"Clone Repository"** pour récupérer ce repository sur ma machine.
Initialement, j'avais installé manuellement les outils nécessaires, notamment **PHP 8.2**. Cependant, j'ai ensuite repris le projet en l'initialisant avec **Docker** (outil de conteneurisation) dans le **WSL Debian** (Windows Subsystem Linux) pour assurer une meilleure gestion des dépendances et de l'environnement de développement.

- **Installation de Laravel**

En suivant le *Quick Start* de la documentation officielle, j'ai installé Laravel en exécutant la commande : `composer create-project laravel/laravel .`

- **Configuration de Docker et WSL**

J'avais déjà installé **Docker Desktop**, donc pour démarrer mon environnement de travail, je dois :
-> **Lancer Docker Desktop**
-> **Ouvrir un terminal PowerShell** et exécuter la commande : `wsl -d debian`
-> **Naviguer vers mon projet** avec : `cd ~/projects/laravel-project-arcadia`
-> **Démarrer les conteneurs Docker** en utilisant : `make up`
Ou si besoin d'un redémarrage complet : `make restart`
*(Ces commandes sont définies dans mon fichier **Makefile** pour simplifier la gestion des conteneurs.)*

J'aurais pu utiliser Git en lignes de commandes et cloner mon repo avec :
« `git clone le_lien_de_mon_projet` »
-> Me déplacer dans le dossier de mon projet : `cd laravel-project-arcadia`
-> Initialiser le projet Laravel dans ce répertoire : `composer create-project laravel/laravel .`
-> Versionner les fichiers une fois Laravel installé : `git add .`
Le reste ne change pas.
-> Faire le premier commit : `git commit -m " first commit: web.php, HomeController, images, css, main route"`
-> Envoyer les modifications sur mon repo GitHub : `git push -u origin main`
-> Pour mieux organiser mon développement, j'ai suivi une stratégie de branches : `git checkout -b branch-name`
-> Après chaque avancée j'ajoute les fichiers, commit et push :

```
git add .
git commit -m "nom du message"
git push origin feat/branch-name
```
-> Une fois les développements terminés et validés, je me place sur la branche du dessous (`main` si le cas est présent) et je fusionne la branche :
```
git chekout main
git merge feat/branch-name
git push origin main
```
Et je recommence à chaque branche/ chaque feature / chaque merge.

Développement de la partie front-end de l'application Zoo Arcadia
Création des éléments graphiques :

- Utilisation de Canva pour créer le **logo** (*Cf. Annexes, Activité Type 1, Exemple 1, Pic. 1a et 1b*).
- Utilisation de Figma pour élaborer la **charte graphique** (*Cf. Annexes, Activité Type 1, Exemple 1, Pic. 1c*), ainsi que les maquettes **Wireframes** (*Cf. Annexes, Activité Type 1, Exemple 1, Pic. 2a à Pic. 2c : mobile, et Pic.4a à Pic.4c : desktop)* et **Mockup** (*Cf. Annexes, Activité Type 1, Exemple 1, Pic. 3a à Pic. 3c : mobile ; Pic. 5a à Pic. 5c : desktop*).

Développement des interfaces utilisateur :

- Création des interfaces pour les pages principales : Accueil, Animaux, Habitats, Services, Avis, et Contact.
- Structuration du contenu avec HTML5 (pages .blade avec Laravel *: voir page 6 "**Précisez les moyens utilisés**" pour **l'installation de Laravel**.*) en respectant les standards d'accessibilité pour garantir une expérience utilisateur inclusive.
- Utilisation de balises sémantiques comme **<header>**, **<main>**, et **<footer>** : layouts/base.blade pour améliorer l'accessibilité et le SEO. (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.6)*
- Ajout d'éléments interactifs tels que des boutons pour naviguer entre les pages et des formulaires pour collecter des données utilisateur (ex. avis, contact).

Mise en forme et personnalisation des styles :

- Utilisation de CSS3 (SCSS) pour assurer un design **responsif** et adapté à tous types d'écrans avec les media queries. (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.7)*
- Création de fichiers pour les variables (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.8). Affichage en quinconce des cartes qui affichent les services avec la pseudo-classe :nth-child* (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.9)* les composants comme les boutons, le menu format mobile (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.10)*.

Intégration des interactions utilisateur :

- Utilisation de JavaScript pour des fonctionnalités dynamiques, telles que l'affichage conditionnel et la validation des formulaires.
- Validation des formulaires côté client avant l'envoi des données, pour guider l'utilisateur (par exemple, afficher un message d'erreur si un champ est vide).
- Assurance d'une navigation fluide grâce à un menu intuitif permettant d'accéder rapidement aux différentes sections du site.

Utilisation de JavaScript pour la partie dynamique :

- Utilisation de la méthode **addEventListener()** de la classe **Event.Target** pour des fonctionnalités

comme le carrousel.
- Récupération des données avec l'API (Application programming interface) fetch en utilisant des fonctions asynchrones. (*Cf. Annexes, Activité Type 1, Exemple 1, Pic.11 à 13, pages 26-27)*
- Orientation des choix d'interface basée sur les retours des utilisateurs finaux imaginaires (personas).

2. Précisez les moyens utilisés :

Pour ce projet j'ai utilisé :
Le logiciel Figma pour la réalisation des maquettes
L'outil de design et de communication visuelle en ligne Canva pour le logo.
L'IDE PhpStorm de la suite JetBrains pour un développement efficace.
Frontend :
- HTML5, CSS3, JS

3. Avec qui avez-vous travaillé ?

Travail réalisé en autonomie dans un cadre individuel. Respect des délais impartis dans le cadre de l'ECF.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *Projet ECF donné par Studi*

Chantier, atelier, service ▸ Formation

Période d'exercice ▸ Du : *01/07/2024* au : *22/11/2024*

5. Informations complémentaires *(facultatif)*

Le projet **Arcadia** est une application web prévue pour l'ECF
Le projet a été développé dans le cadre de la formation DWWM afin de valider les compétences du référentiel en développement web front-end et back-end.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ▸ ***Mini jeu parallaxe Yoshi en JavaScript***

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour ce projet, j'ai reproduit un mini-jeu Mario en parallaxe en utilisant JavaScript et CSS, mais en remplaçant Mario par Yoshi et en créant mon propre décor.

Recherche et préparation des éléments graphiques :

- Recherche d'une planche de personnages en 2D sur internet pour créer le sprite (succession d'images matricielles) de Yoshi en marche.
- Rognage des parties nécessaires pour obtenir 25 images consécutives représentant le mouvement de Yoshi.
- Utilisation de Figma pour créer les éléments de décor tels que les arbres, les herbes, le ciel et les nuages *(Cf. Annexes, Activité Type 1, Exemple 2, Pic. 14)*.

Création du fichier HTML :

- Création d'un fichier HTML avec des ** ** identifiées par des classes pour accueillir les différentes parallaxes *(Cf. Annexes, Activité Type 1, Exemple 2, Pic. 15a)*.

Mise en forme avec CSS :

- Réinitialisation des marges avec un reset CSS.
- Positionnement de Yoshi en absolu pour un placement précis sans être affecté par les autres éléments de la page.
- Utilisation de la propriété **animation** sur les classes **.yoshi-left** et **.yoshi-right** avec la fonction **steps(25)** pour créer un effet de marche à la vitesse de 3 secondes.
- Ajout de la propriété **transform: scaleX(-1)** pour inverser l'animation vers la gauche.
- Positionnement des éléments de décor pour créer un effet de répétition infinie.
- Utilisation de **keyframes** pour créer un effet de mouvement plus ou moins lent, simulant le défilement du paysage.

Gestion des événements :

- Affichage d'une modal au chargement de la fenêtre avec une description de l'utilisation de l'application *(Cf. Annexes, Activité Type 1, Exemple 2, Pic. 15b et 15c).*
- Fermeture de la modal en cliquant sur le bouton fermer (X) ou n'importe où dans la fenêtre avec **window.click**, cette action ne se produisant qu'une seule fois grâce à l'option **{ once: true }**.

Interactions utilisateur :

- Utilisation des événements clavier pour déplacer Yoshi :
  - **keydown** (marche) : Ajout de la classe **.yoshi-left** ou **.yoshi-right** pour changer la direction et appliquer l'animation, ainsi que la mise à jour de la variable **move** pour modifier la position du paysage.
  - **keyup** (arrêt) : Retrait des classes **.yoshi-left** et **.yoshi-right** pour arrêter les animations de marche lorsque la touche est relâchée.

2. Précisez les moyens utilisés :

Pour ce projet, j'ai utilisé un peu de HTML, du CSS et du JS vanilla.
J'ai choisi l'IDE PhpStorm.
Avec Figma j'ai créé le sprite du personnage Yoshi et tout le décor.

Déploiement sur Netlify : https://yoshi-parallax.netlify.app

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule pour ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *Projet personnel*

Chantier, atelier, service ▸ Formation

Période d'exercice ▸ Du : *28/12/2024* au : *15/01/2025*

5. Informations complémentaires *(facultatif)*

Évolutions futures : Yoshi pourra sauter par-dessus et marcher sur les tonneaux, ajout d'ennemis. Ajout du format mobile (des touches flèches ou une fonction au toucher de l'écran qui ferait avancer le personnage)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n° 1 ▸ ***Zoo Arcadia***

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Avant de pouvoir coder la partie back-end, je réalise le **MCD** (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.18*) ainsi que les **diagrammes de cas d'utilisation** (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.19*) et le **diagramme de séquence** pour le login (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.20*) à l'aide du logiciel FigJam (Figma).

Je peux maintenant me concentrer sur la partie back-end de l'application Zoo Arcadia, développée avec le framework Laravel pour gérer les fonctionnalités administratives et les données du zoo de manière sécurisée et performante.

En utilisant la commande **php artisan make:migration create_habitats_table**, je crée la table des habitats et la complète dans la migration (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.21*). Cela me permet ensuite de créer des seeders, qui me fourniront des entrées dans la base de données en local, évitant ainsi de réécrire les données à chaque modification ou suppression lors des tests de mes fonctionnalités.

J'ai porté une attention particulière à la sécurité des accès grâce à une gestion précise des rôles et des permissions avec la façade Gate – Policy (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.22*).
Concernant les fonctionnalités, le back-end permet une gestion complète des utilisateurs. Un système d'authentification sécurisé a été mis en place pour limiter l'accès aux seuls administrateurs, vétérinaires et employés, empêchant les visiteurs de créer des comptes (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.23*). Chaque rôle dispose de permissions spécifiques : l'administrateur gère les comptes, habitats, services et animaux ; les vétérinaires saisissent des rapports médicaux sur les animaux et ajoutent des commentaires sur les habitats ; les employés valident les avis des visiteurs, modifient les services proposés et répondent aux messages reçus via le formulaire de contact.

La gestion des données du zoo est un élément essentiel. Par exemple, pour les habitats des animaux, il est possible d'ajouter, consulter, modifier ou supprimer des informations grâce à un système appelé **CRUD** (Create, Read, Update, Delete).

- **Créer un habitat (Create)** : Cette action permet d'ajouter un nouvel habitat. Les informations sont saisies via un formulaire validé (*HabitatsFormRequest*) avant d'être enregistrées dans la base de données. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.24)*
- **Consulter les habitats (Read)** : Il existe plusieurs façons de lire les informations des habitats :
  - *index* : Affiche la liste complète des habitats. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.25)*
  - *show* : Permet d'afficher les détails d'un habitat précis en fonction de son nom. Si

l'habitat n'existe pas, un message d'erreur est affiché. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.26)*
    - *habitats* : Montre simplement la page des habitats, sans données spécifiques. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.27)*
- **Modifier un habitat (Update)** : Si un habitat doit être mis à jour, on utilise deux étapes :
    - *edit* : Récupère les informations d'un habitat spécifique et affiche un formulaire de modification. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.28)*
    - *update* : Enregistre les nouvelles informations après validation du formulaire. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.29)*
- **Supprimer un habitat (Delete)** : Cette action permet d'effacer un habitat en sélectionnant son nom. *(Cf. Annexe, Activité Type 2, Exemple 1, Pic.30)*

Grâce à ce système, la gestion des habitats est simplifiée et organisée, facilitant le travail des responsables du zoo.

La **sécurité** a été une priorité tout au long du développement. Les mots de passe sont hashés avec **Bcrypt** automatiquement par Laravel et les permissions sont définies à l'aide des **Policies** et **Gates** (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.31, 32 et 33*) pour garantir que chaque utilisateur n'accède qu'aux fonctionnalités qui lui sont autorisées. Les formulaires incluent des tokens **Cross-Site-Request-Forgery** (les jetons @csrf) pour éviter de contourner les droits administrateurs. Également, les identifiants de connexion à la base de données sont protégés dans un fichier **.env** .

D'un point de vue technique, j'ai choisi Laravel pour sa simplicité et sa robustesse. L'utilisation d'Eloquent ORM (Object Relationship Manager) facilite les interactions avec la base de données MySQL, utilisée pour les données structurées comme les utilisateurs ou les habitats. (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.34, 35 et 36) pour visualiser ce que donneraient les requêtes en MariaDB).*
MongoDB, en revanche, a été adopté pour stocker les statistiques de consultation des animaux. (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.37 : Installation MongoDB pour Laravel, et Pic 38, 39a et 39b : Controller, migration et Model*)

Le stockage des fichiers, comme les images des animaux et habitats, est géré via le système intégré de Laravel (Storage) pour une organisation claire et sécurisée.

Enfin, le projet a été développé dans un environnement configuré avec l'IDE PHPStorm, ce qui a permis un flux de travail fluide et efficace. L'application a été déployée sur un VPS Debian via Docker, (*Cf. Annexe, Activité Type 2, Exemple 1, Pic.40 à 43 : Dockerfile, docker-compose, mongo-init et Makefile)* bien que quelques défis aient été rencontrés lors de la configuration, notamment avec Nginx, les domaines SSL et le protocole sécurisé https.
Pour les tests, des scénarios manuels ont permis de simuler les actions des différents utilisateurs.

2. Précisez les moyens utilisés :

Pour réaliser Arcadia ainsi que chacun de mes projets, j'ai **configuré mon environnement de travail** comme suit :

- **Système et outils de base :**
  - **Système d'exploitation** : WSL Debian sous Windows
  - **IDE** : PhpStorm ou l'éditeur de code VS Code avec extensions Laravel, React et Tailwind
  - **Gestionnaire de versions** : Git / GitHub pour le suivi de projet.
- **Installation des outils nécessaires**
  - **Docker** : Pour conteneuriser le projet
  - **WSL et dépendances :** Curl
  - **Gestionnaires de paquets :** Composer pour PHP, Pnpm pour JavaScript (seulement Arcadia)

(Cf. Annexe, Pic.44 à Pic.46, page 45)

- **Clone du dépôt Git**
- **Lancer l'environnement avec Docker**
- **Installer les dépendances** : Backend Laravel (make api). Frontend Vite (make pnpm cmd= "install")
- **Configuration des fichiers spécifiques** : Variables d'environnement (.env). Configuration Nginx (defaut.conf)
- **Accès à la base de données** avec AlwaysData => Arcadia. (Docker -> PostgreSQL pour d'autres projets)

Le software Figjam (Figma) pour les **diagrammes de cas d'utilisation**, **diagramme de séquence** et Excalidraw pour faire le **MCD** *(Modèle Conceptuel de Données)* (*Cf. Annexes, Activité Type 2, Exemple1, Pic.18, 19 et 20*).

Backend :

- PHP : Framework Laravel

Base de données :

- SQL : AlwaysData MariaDB
- NoSQL : MongoDB

Déploiement :
Hebergement sur VPS debian chez OVH : XXX

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *Projet ECF donné par Studi*

Chantier, atelier, service ▸ Formation

Période d'exercice ▸ Du : *01/07/2024* au : *22/11/2024*

5. Informations complémentaires *(facultatif)*

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n° 2 ▸ ***QuizApp***

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'application QuizApp a été réalisée dans le cadre de ma formation DWWM.

Tout d'abord, j'ai initialisé mon repository sur GitHub, puis j'ai cloné mon projet sur PHPStorm afin de commencer à structurer mon application. *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.47 – Structure du projet)*

Puisque j'ai travaillé en PHP vanilla, j'ai organisé mon projet avec :

- un répertoire **classes/** contenant mes classes PHP (*Autoloader, Database, Question et Quiz*),
- un répertoire **public/** destiné aux fichiers CSS et JavaScript.

Structure du code :
J'ai créé un **Autoloader** avec la méthode register() qui enregistre autoload().
Ainsi, dès qu'une classe est instanciée, **PHP** appelle automatiquement autoload(), ce qui permet de récupérer les classes sans avoir à les inclure manuellement à chaque fois avec require_once.

La classe **Database** gère la connexion à la base de données et assure qu'elle n'est instanciée qu'une seule fois, ce qui évite d'écrire new PDO() à plusieurs endroits dans le code.

La classe **Question** permet de récupérer les **questions** et leurs **réponses associées** depuis la base de données, en utilisant la connexion fournie par la classe **Database**.

Enfin, la classe **Quiz** vérifie les réponses d'un utilisateur et calcule son score.

Intégration de la base de données :
Pour intégrer ma **base de données** à mon application, j'ai choisi **AlwaysData**. *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.48 – Connexion à la base de données MariaDB via AlwaysData sur PHPStorm)*
Une fois ma base de données prête, je me place dessus :
i-jurain_quiz
Puis, je fais un **clic droit** sur ma base et j'ouvre une nouvelle console :
New -> Query Console

Je peux alors créer mes tables *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.49)* et exécuter mes requêtes via la console SQL de **PHPStorm**. *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.50)*

Pour éviter une erreur lors du lancement du script, j'exécute la requête :
'DROP TABLE IF EXISTS [nom de la table];'

Cela empêche un message d'erreur indiquant que la table existe déjà. *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.51)*

Exemple de requête de jointure (JOIN) et la suppression de la réponse de cette requête (DELETE). *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.52)*

Insertions dans les tables *questions* et *answers*. *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.53)*

Mise à jour des données avec UPDATE *(Cf. Annexe, Activité Type 2, Exemple 2. Pic.54)*
Pour cela, j'exécute deux requêtes SQL :

1. La première met à jour la réponse Blanche en indiquant qu'elle est **fausse** (booléen = 0).
2. La seconde met à jour la réponse Grise en indiquant qu'elle est **correcte** (booléen = 1)

(Cf. Annexe, Activité Type 2, Exemple 2. – Mon script SQL complet en Pic.55 et une capture du QuizApp en Pic.56)

2. Précisez les moyens utilisés :

Ce projet a été réalisé avec mon IDE PHPStorm, AlwaysData pour créer ma base de données.
Hôte MySQL mysql-i-jurain.alwaysdata.net
Version : 10.11 (mariadb)

Stack technique :
PHP vanilla (POO)
MySQL
HTML/CSS
JavaScript vanilla
Composer

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule sur ce projet.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ▸ *Projet personnel*

Chantier, atelier, service ▸ Formation

Période d'exercice ▸ Du : *06/03/2025* au : *10/03/2025*

5. Informations complémentaires *(facultatif)*

Cette application de Quiz a été développée pour m'exercer avec la base de données.

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e) Isabelle BEAUREGARD,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à Paris le 19/01/2025

pour faire valoir ce que de droit.

Signature :

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

ANNEXES

Activité Type 1
Exemple 1
Logo desktop et mobile :

Pic.1a

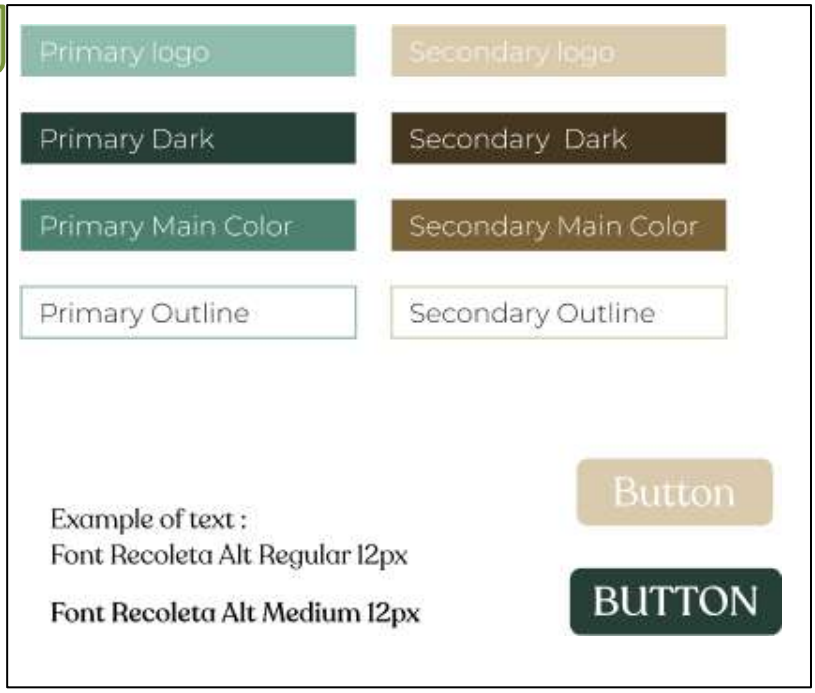


Pic.1b



Charte graphique

Pic.1c

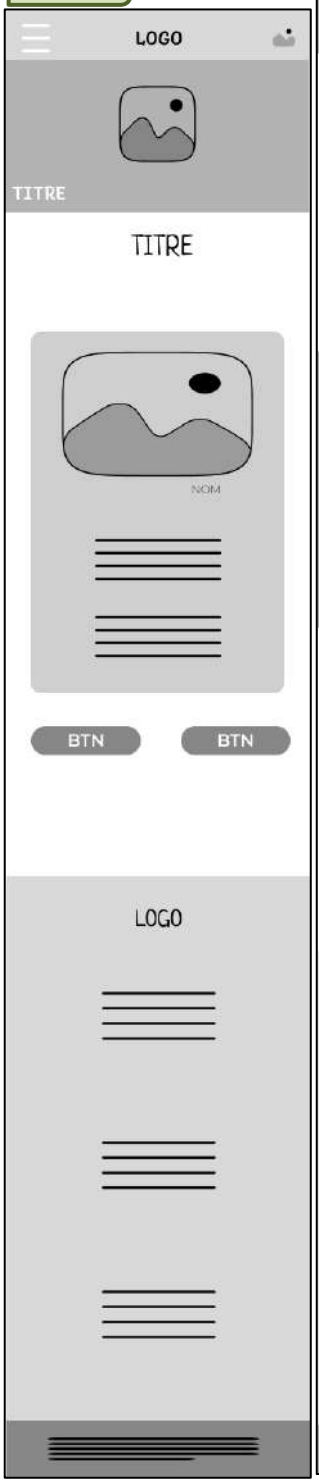


Exemples de wireframes et de mockup mobile :

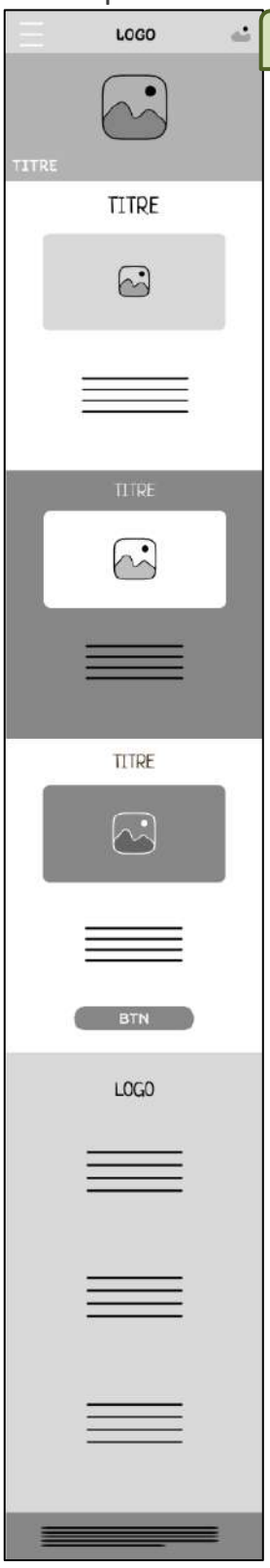
Pic.2a



Pic.2b



Pic.2c



Pic.3a



Pic.3b



Pic.3c

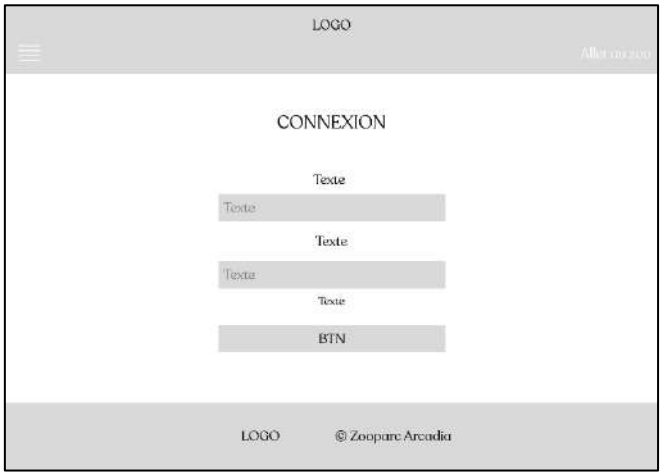


Exemples de wireframes et mockup desktop :

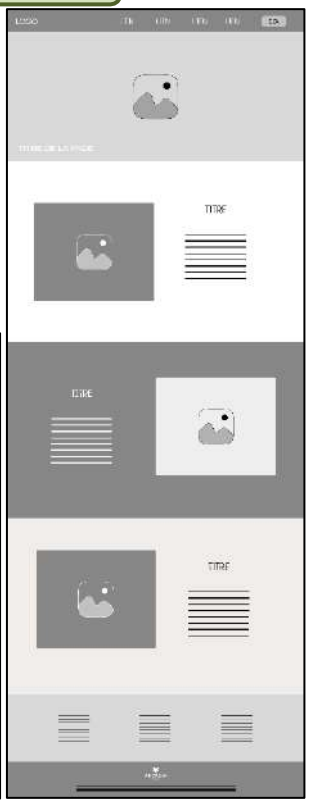
Pic.4a



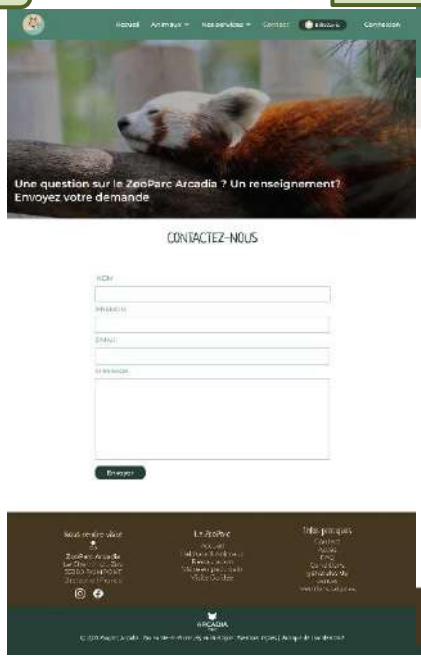
Pic.4b



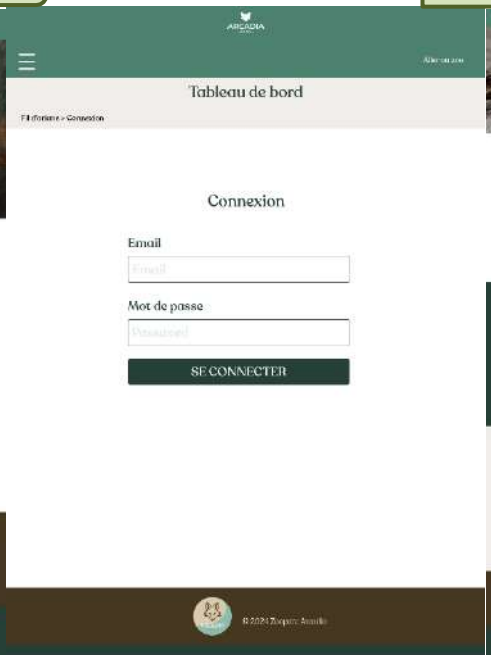
Pic.4c



Pic.5a



Pic.5b



Pic.5c



Extrait de code layouts/base.blade : HTML, balises

Pic.6

```
<!doctype html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible"
          content="ie=edge">
    <title>@yield('title', 'Welcome') - Zoo Arcadia</title>
    @vite(['resources/css/app.scss', 'resources/js/app.js'])
</head>
<body>
<!-- ~~~~~~~~~~~~~~~~~~~~~~~~~~ HEADER ~~~~~~~~~~~~~~~~~~~~~~~~~~ -->
<header>
    <div class="div-header">
        <nav>
            <a href="#">
                <img alt="Menu burger" class="menu-burger" src="/asset/icons/white-menu-burger.svg">
            </a>
            <a href="{{ route('home') }}">
                <img alt="logo arcadia mobile" class="logo-header" src="/asset/logos/logo-footer.svg"/>
                <img src="/asset/logos/logo-header.svg" class="logo-desktop"
                     alt="logo arcadia desktop">
            </a>
            <a href="{{ route('ticketing') }}">
                <img alt="icon ticket" class="icon-ticket" src="/asset/icons/icon-ticket.svg"/>
            </a>
        </nav>
        <div class="nav-links">
            <ul id="nav-link">
                <li><a href="{{ route('animals') }}">Animaux</a></li>
                <li><a href="{{ route('habitats')}}">Habitats</a></li>
                <li><a href="{{ route('services')}}">Services</a></li>
                <li><a href="{{ route('contact')}}">Contact</a></li>
                <li><a href="{{ route('ticketing')}}">Billetterie</a></li>
            </ul>
        </div>
    </div>
</header>

<main>
    <!-- ~~~~~~~~~~~~~~~~~~~~~~~~~~ Section background header title ~~~~~~~~~~~~~~~~~~~~~~~~~~ -->
    <section id="bg-header">
        @yield('header')
    </section>
    @include('partials.flashMessage')
    @yield('content')


</main>
<!-- ~~~~~~~~~~~~~~~~~~~~~~~~~~ NAV FOOTER ~~~~~~~~~~~~~~~~~~~~~~~~~~ -->
<section class="container-navfooter">
    <div class="navfooter-content">
        <a href="{{ route('home')}}"><img alt="Le logo d'Arcadia"
                        class="logo-footer"
                        src="/asset/logos/logo-header.svg">
        </a>

        <div class="navfooter-location">
            <h3>Nous rendre visite</h3>
            <img alt="Icône de localisation"
                 class="icon-footer"
                 src="/asset/icons/map.svg">
            <ul>
                <li><em>ZooParc Arcadia</em></li>
                <li>123 le Chemin de Brocéliande</li>
                <li>35380 PAIMPONT</li>
                <li>BRETAGNE | FRANCE</li>
            </ul>
        </div>

        <div class="navfooter-website">
            <h3>Plan du site</h3>
            <ul>
                <li><a href="{{ route('home')}}">Accueil</a></li>
                <li><a href="{{ route('habitats') }}">Habitats</a></li>
                <li><a href="{{ route('animals') }}">Animaux</a></li>
                <li><a href="{{ route('services')}}">Services</a></li>
                <li><a href="{{ route('review')}}">Avis</a></li>
            </ul>
        </div>

        <div class="navfooter-infos">
            <h3>Infos Pratiques</h3>
            <ul>
                <li><a href="{{ route('contact')}}">Contact</a></li>
                <li><a href="{{ route('legal-notices')}}">Mentions légales</a></li>
                <li><a href="{{ route('login')}}">Compte Pro</a></li>
            </ul>
        </div>

    </div>
</section>
<!-- ~~~~~~~~~~~~~~~~~~~~~~~~~~ FOOTER ~~~~~~~~~~~~~~~~~~~~~~~~~~ -->
<section class="container-footer">
    <h4>&copy; Zooparc Arcadia : Zoo en Ille et vilaine (35) en Bretagne</h4>
    <span>Application en Laravel réalisée par                    ) - ECF École Studi</span>
</section>

</body>
</html>
```

Extraits de code : Media Queries CSS

Pic.7

```css
/* ---------------------------------- MEDIA QUERIES ----------------------------------*/
@media (min-width: 768px) {
    .title-animals {
        font-size: 2.2rem;
    }

    .div-services {
        flex-direction: row;
        text-align: left;
        max-width: 100%;
    }
    .content-animal-sheet {
        flex-direction: row;
        align-items: flex-start;
        gap: 50px;
    }

    .img-modal {
        flex: 0 0 45%;
    }

    .description-animal {
        flex: 1;
        padding-top: 0;
    }

    /* Section impaire : image à gauche, texte à droite */
    .container-services:nth-child(odd) .img-services {
        order: 1;
        flex: 1;
        padding-right: 20px;
    }

    .container-services:nth-child(odd) .description-services {
        order: 2;
        flex: 1;
        padding-left: 20px;
    }

    /* Section paire : texte à gauche, image à droite */
    .container-services:nth-child(even) .img-services {
        order: 2;
        flex: 1;
        padding-left: 20px;
    }

    .container-services:nth-child(even) .description-services {
        order: 1;
        flex: 1;
        padding-right: 20px;
    }

    .image-services {
        width: 100%;
        height: auto;
        border-radius: 8px;
    }
    .card-animals-habitat{
        display: grid;
        grid-template-columns: repeat(4, 1fr); /* 4 colonnes de taille égale */
        grid-template-rows: repeat(2, auto);
        gap: 3rem;
    }

    .animals-container {
        display: grid;
        grid-template-columns: repeat(4, 1fr); /* 4 colonnes de taille égale */
        grid-template-rows: repeat(2, auto);
        gap: 3rem;
    }
    .animal-card img {
        width: 100%;
        height: 180px;
        object-fit: cover;
        border-radius: 10px;
        margin-bottom: 10px;
    }
}

@media (min-width: 488px) and (max-width: 518px) {
    .animals-container {
        display: grid;
        grid-template-columns: repeat(2, 1fr); /* 4 colonnes de taille égale */
        grid-template-rows: repeat(2, auto);
        gap: 3rem;
    }
}
```

Extraits de code : Variables CSS (Pic.8)

Pic.8

```css
:root {
    --primary-xlight: #8fbcac88;
    --primary-light: #8fbcac;
    --primary: #4b816e;
    --primary-dark: #264037;
    --bg-primary-xlight: #EFF5F3;

    --secondary-light: #dacbaf;
    --secondary: #7a6238;
    --secondary-dark: #463820;
    --bg-primary-light: #D0E1DB;
    --bg-secondary-light: #f0edea;

    --light: #fff;
    --dark: #1a1a1a;

    --bg-light: #f2f2f2;
    --bg-card: #17257573;

    --color-studi: #49a9b4;

    --success: #10b981;
    --info: #0ea5e9;
    --warning: #ff8330;
    --danger: #fd3333;

    --btn-primary: #e5e5e5;
    --btn-secondary: #595959;

    --font-primary: "Recoleta Alt";
    --font-secondary: "LXGWWenKaiTC-Regular";
    --font-title: "KleeOne-Regular";

    /* --------- 0.5rem = 8px | 1rem = 16px --------- */
    --h1-font-size: 3rem;
    --h2-font-size: 1.5rem;
    --h3-font-size: 1.25rem;
    --normal-font-size: 1rem;
    --small-font-size: 0.5rem;
}
```

Extraits de code : CSS page Services (Pic.9) CSS navigation menu burger (Pic.10)

Pic.9

```css
/* Section des services */
.container-services {
    display: flex;
    flex-direction: column;
    align-items: center;
    margin: 20px;
}

.div-services {
    display: flex;
    flex-direction: column;
    align-items: center;
    margin-bottom: 30px;
    text-align: center;
    max-width: 600px;
}

.title-services {
    font-size: 2.2em;
    text-align: center;
}

.img-services {
    position: relative;
}

.image-services {
    width: 100%;
    border-radius: 8px;
    margin-top: 10px;
}

.description-services {
    margin-top: 15px;
    padding: 0 15px;
}

.p-services {
    color: #333;
    font-size: 1em;
    line-height: 1.6;
}

/* Styles paire */
.container-services:nth-child(odd) {
    color: var(--secondary-dark);
}

/* impaire */
.container-services:nth-child(even) {
    background-color: var(--primary);
    color: white;
    padding: 20px;
    border-radius: 8px;
}

.container-services:nth-child(even) .p-services {
    color: var(--light);
}
```

Pic.10

```css
.nav-links {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(38, 64, 55, 0.9);
    backdrop-filter: blur(5px);
    z-index: 100;
    justify-content: center;
    align-items: center;
    text-align: center;
}

.nav-links ul {
    padding: 0;
}

.nav-links ul li {
    margin: 1rem 0;
}

.nav-links ul li a {
    font-size: 1.5rem;
    color: white;
    transition: color 0.3s ease;
}

.nav-links ul li a:hover {
    text-decoration: underline;
}

.logo-desktop{
    display: none;
}

.txt-hidden{
    color: transparent;
}

.menu-open .nav-links {
    display: flex;
}

@media (min-width: 1024px) {
    .menu-burger {
        display: none;
    }
    img.logo-header{
        display: none;
    }
    .logo-desktop{
        display: flex;
        justify-content: center;
    }

    .nav-links {
        display: flex;
        position: static;
        height: auto;
        width: auto;
        background-color: transparent;
        backdrop-filter: none;
        flex-direction: row;
        justify-content: flex-end;
        align-items: center;
    }

    .nav-links ul {
        display: flex;
        gap: 2rem;
        padding: 0;
    }

    .nav-links ul li {
        margin: 0;
    }

    .navfooter-content{
        display: flex;
        align-items: center;
        justify-content: space-around;
    }

    .logo-header {
        margin-right: auto;
    }
}
```

Extraits de code : carousel.js | reviewsForm.js | sendMail.js (Fetch Pic.11, 12 et 13)

Pic.11

```
let error = ""
const carouselContent = document.querySelector(".carousel-content")
const section = document.querySelector(".section-reviews")
const carousel = document.querySelector(".carousel")
const errorContainer = document.getElementById("error-container")
let currentIndex = 0
const btnPrevious = document.querySelector(".btn-previous")
const btnNext = document.querySelector(".btn-next")

let reviews = []

async function getReviews() {
    try {
        const response = await fetch("/api/admin/zoo/reviews")
        reviews = await response.json()
        makeCarousel(reviews, currentIndex)
    } catch (e) {
        errorContainer.textContent = `Erreur: ${e.message}`
        carouselContent.innerHTML = ""
    }
}

function makeCarousel(reviews, index) {
    if (reviews.length === 0) {
        carouselContent.innerHTML = "<p>Aucun avis disponible.</p>"
        return
    }

    carouselContent.innerHTML = ""

    const title = document.createElement("h3")
    title.textContent = reviews[index].author

    const rating = document.createElement("p")
    rating.textContent = `Rating: ${reviews[index].rating}`

    const content = document.createElement("p")
    content.textContent = reviews[index].content

    carouselContent.appendChild(title)
    carouselContent.appendChild(rating)
    carouselContent.appendChild(content)
}

function previous() {
    if (currentIndex <= 0) {
        currentIndex = reviews.length - 1
    } else {
        currentIndex -= 1
    }
    makeCarousel(reviews, currentIndex)
}

function next() {
    if (currentIndex >= reviews.length - 1) {
        currentIndex = 0
    } else {
        currentIndex += 1
    }
    makeCarousel(reviews, currentIndex)
}

btnPrevious.addEventListener("click", previous)
btnNext.addEventListener("click", next)

void getReviews()
```

Pic.12

```
const contactForm = document.querySelector('.form-contact-us')

async function handleSubmit(event) {
    event.preventDefault()

    const formData = new FormData(contactForm)
    const contactData = Object.fromEntries(formData)
    const response = await fetch("/api/contact", {
        method: 'POST',
        headers: {
            "Accept": "application/json",
            "Content-Type": "application/json",
        },
        body: JSON.stringify(contactData),
    })

    if (response.ok) {
        location.reload()
    }
}

contactForm.addEventListener('submit', handleSubmit)
```

Pic.13

```
const form = document.querySelector('.form')

async function handleSubmit(event){
    event.preventDefault()
    const reviewData = Object.fromEntries(new FormData(form))

    const response = await fetch("/api/admin/zoo/reviews/create", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
            "Accept": "application/json",
        },
        body: JSON.stringify(reviewData)
    })
    if(response.ok){
        location.reload()
    }
}

form.addEventListener('submit', handleSubmit)

document.querySelector('.menu-burger').addEventListener('click', function(e) {
    e.preventDefault();
    document.querySelector('.nav-header').classList.toggle('menu-active');
});
```

Activité Type 1

Exemple 2

Pic.14



Pic.15a

index

```
1  <!DOCTYPE html>
2  <html lang="fr">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <link rel="stylesheet" href="css/main.css" />
7      <script src="js/main.js" defer></script>
8      <title>Jeu Parallax</title>
9    </head>
10   <body>
11     <div class="landscape"></div>
12     <div class="yoshi"></div>
13
14     <div id="modal" class="modal">
15       <div class="modal-content">
16         <span class="close-btn" id="close-btn">×</span>
17         <p>Cliquez n'importe où pour écouter la musique ;)</p>
18         <p>Déplacez-vous avec les flèches directionnelles de votre clavier.</p>
19         <p>Amusez-vous bien ☺</p>
20         <p class="name">Isabelle ♥</p>
21       </div>
22     </div>
23   </body>
24 </html>
```

Extrait de code : Modal JS et CSS

Pic.15b

```js
const modal = document.getElementById("modal")
const closeBtn = document.getElementById("close-btn")

window.addEventListener(
  "click",
  () => {
    const audio = new Audio(
      "./audio/SuperMario64Soundtrack-PirahnaPlantsLullaby.mp3"
    )
    audio.play().catch((err) => console.error("Erreur audio :", err))

    modal.style.display = "none"
  },
  { once: true }
)

closeBtn.addEventListener("click", () => {
  modal.style.display = "none"
})

window.addEventListener("load", () => {
  modal.style.display = "flex"
})

const yoshi = document.querySelector(".yoshi")
const landscape = document.querySelector(".landscape")
const sky = document.querySelector("body")

let move = 0

window.addEventListener("load", () => {
  document.addEventListener("keydown", walk)
  document.addEventListener("keyup", stop)
})

function walk(event) {
  switch (event.key) {
    case "ArrowLeft":
      yoshi.classList.add("yoshi", "yoshi-left")
      yoshi.classList.remove("yoshi-right")
      move++
      parallax()
      break

    case "ArrowRight":
      yoshi.classList.add("yoshi", "yoshi-right")
      yoshi.classList.remove("yoshi-left")
      move--
      parallax()
      break
  }
}

function stop(event) {
  yoshi.classList.remove("yoshi-right")
  yoshi.classList.remove("yoshi-left")
}

function parallax() {
  landscape.style.backgroundPositionX = move * 4 + "px"
  sky.style.backgroundPositionX = move + "px"
}
```

Pic.15c

```css
* {
  margin: 0;
  padding: 0;
}

body {
  background: url("../images/sky.svg") repeat-x;
  background-size: auto 100vh;
}

.landscape {
  background: url("../images/landscape.svg") repeat-x;
  position: fixed;
  bottom: 0;
  height: 406px;
  width: 100%;
}

.yoshi {
  background: url("../images/sprite-yoshi-right.svg");
  height: 70px;
  width: 45.28px;
  bottom: 65px;
  position: absolute;
  left: 50%;
}

.yoshi-left {
  animation: yoshi 3s steps(25) infinite;
  transform: scaleX(-1);
}

.yoshi-right {
  animation: yoshi 3s steps(25) infinite;

}

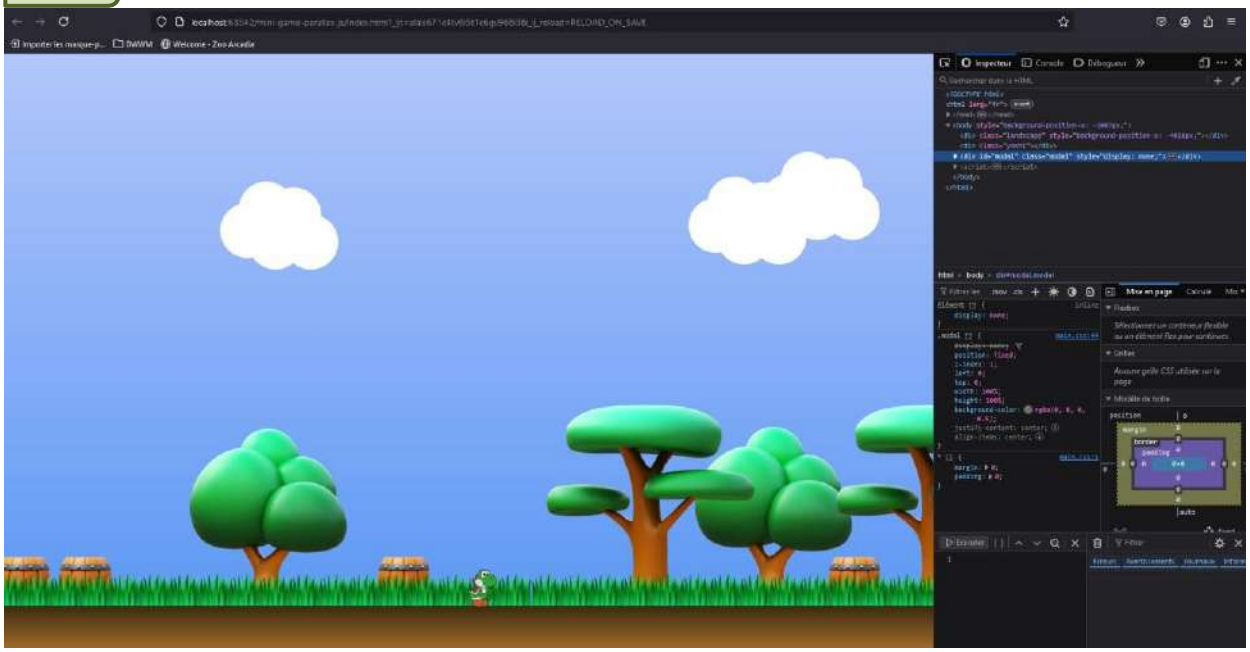
@keyframes yoshi {
  to {
    background-position-x: -1132px;
  }
}

.modal {
  display: none;
  position: fixed;
  z-index: 1;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  justify-content: center;
  align-items: center;
}

.modal-content {
  background-color: white;
  padding: 20px;
  border-radius: 10px;
  text-align: center;
  font-family: "Mochiy Pop One";
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}

.close-btn {
  position: absolute;
  top: 10px;
  right: 20px;
  font-size: 30px;
  cursor: pointer;
}
```

Pic.16

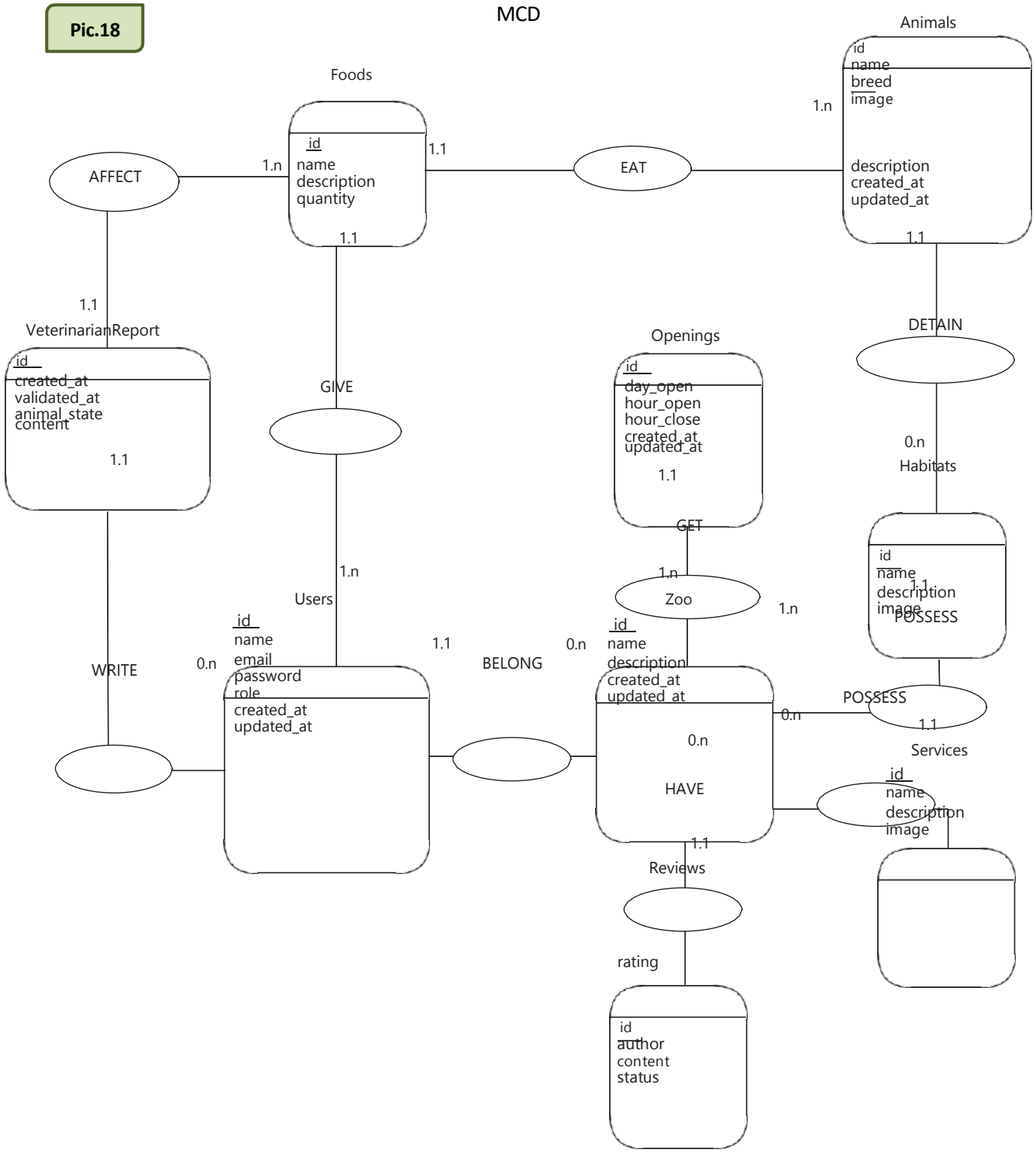


Pic.17


Cliquez n'importe où pour écouter la musique :)
Déplacez-vous avec les flèches directionnelles de votre clavier.
Amusez-vous bien ☺
Isabelle ♥

Activité Type 2
Exemple 1
Pic. 18
MCD

Pic.18



Pic.19

Pic. 19 Diagramme de cas d'utilisation

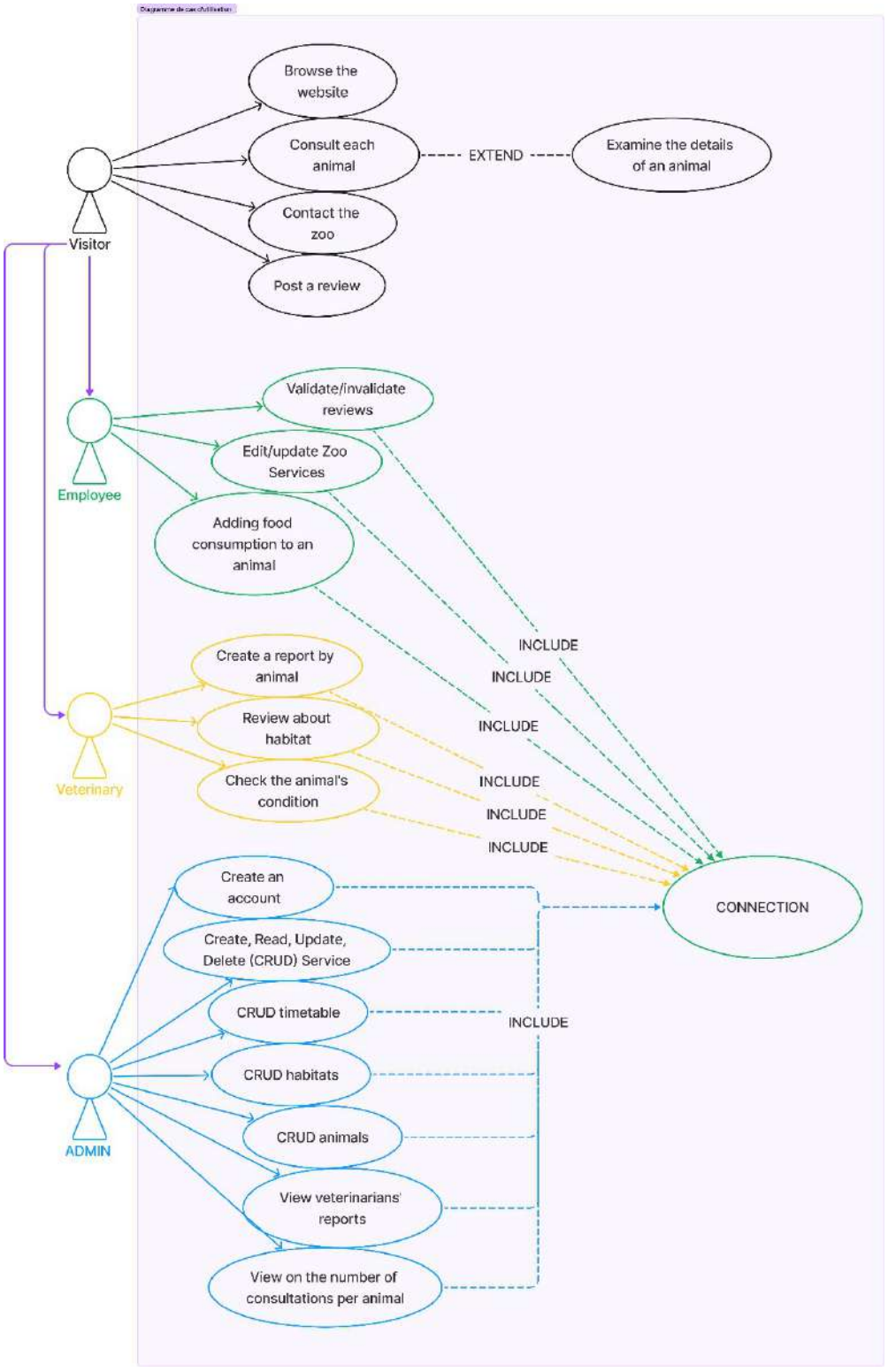


Diagramme de cas d'utilisation (Arcadia)

Pic.20

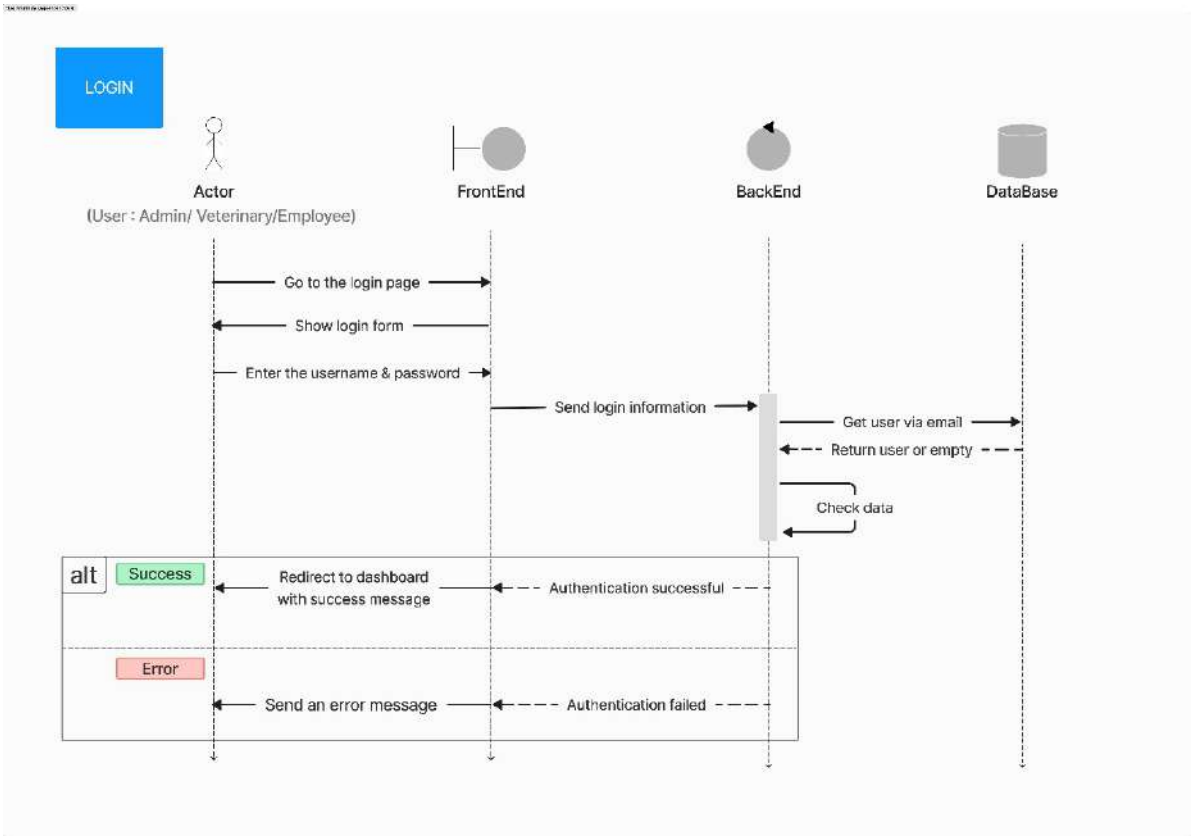


Table "habitats" créée avec la commande **php artisan make :migration create_habitats_table**

Pic.21

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('habitats', function (Blueprint $table) {
            $table->id();
            $table->foreignId('zoo_id'); // ->constrained()->cascadeOnDelete();
            $table->string('name')->unique();
            $table->text('description');
            $table->string('image');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('habitats');
    }
};
```

Extrait de code Façade Gate – Policies :

Pic.22

```php
<?php

namespace App\Policies;

use App\Models\User;
use Illuminate\Support\Facades\Gate;

class HabitatsPolicy extends Gate
{
    public function create(User $user): bool
    {
        return $user->role === 'admin';
    }

    public function update(User $user): bool
    {
        return in_array($user->role, ['admin', 'veterinarian']);
    }

    public function delete(User $user): bool
    {
        return $user->role === 'admin';
    }

    public function view(User $user): bool
    {
        return in_array($user->role, ['admin', 'veterinarian']);
    }
}
```

Pic.23

Extraits de code : Création de l'administrateur (super-admin)

```php
<?php

namespace App\Console\Commands;

use App\Models\User;
use App\Models\Zoo;
use Illuminate\Console\Command;
use Illuminate\Support\Facades\Hash;

class RegisterAdminCommand extends Command
{
    protected $signature = 'register:admin';
    protected $description = 'Register an admin';

    public function handle(): int
    {
        $name = $this->ask('What is your name?');
        $zooName = $this->ask('What is your zoo name?');
        $email = $this->ask('What is your email?');
        $password = $this->secret('What is your password?');

        $validator = validator(
            [
                'name' => $name,
                'zoo_name' => $zooName,
                'email' => $email,
                'password' => $password,
            ],
            [
                'name' => 'required|string',
                'zoo_name' => 'required|string',
                'email' => 'required|email',
                'password' => 'required|string|min:8',
            ]
        );

        if ($validator->fails()) {
            $this->error('Invalid input');
            $this->error($validator->errors());
            return self::FAILURE;
        }

        $admin = new User();
        $admin->zoo_id = Zoo::query()->firstWhere('name', $zooName)->id;
        $admin->name = $name;
        $admin->email = $email;
        $admin->password = Hash::make($password);
        $admin->role = 'admin';

        $admin->save();

        $this->info('Admin registered successfully');

        return self::SUCCESS;
    }
}
```

Pic.24

```php
public function create(HabitatsFormRequest $request): RedirectResponse
{
    $zoo = Zoo::query()->where('name', '=', 'Arcadia')->first();
    $path = $request->file('image')->getClientOriginalName();

    $habitat = new Habitat();
    $habitat->zoo_id = $zoo->id;
    $habitat->name = $request->name;
    $habitat->description = $request->description;
    $habitat->image = $path;

    if (!Storage::disk('public')->exists('asset/images')) {
        Storage::disk('public')->makeDirectory('asset/images');
    }
    Storage::disk('public')->putFileAs("asset/images", $request->file('image'), $path);

    $habitat->save();

    return redirect()->route('home');
}
```

Pic.25

```php
public function index(): View
{
    $habitats = Habitat::all();
    return view('page.habitats', compact('habitats'));
}
```

Pic.26

```php
public function show(string $name): View
{
    $habitat = Habitat::query()->where('name', '=', $name)->first();
    if (!$habitat) {
        throw new Exception("Tu n'as pas d'habitat", 404);
    }
    return view('page.habitat', compact('habitat'));
}
```

Pic.27

```php
public function habitats(): View
{
    return view('page.habitats');
}
```

Pic.28

```php
public function edit(string $name): View
{
    $zoo = Zoo::query()->where('name', '=', 'Arcadia')->first();
    $habitat = Habitat::query()->where('name', '=', $name)->first();

    return view('admin.zoo.habitats.edit', compact('habitat', 'zoo'));
}
```

Pic.29

```php
public function update(HabitatsFormRequest $request, string $name): RedirectResponse
{
    $zoo = Zoo::query()->where('name', '=', 'Arcadia')->first();
    $habitat = Habitat::query()->where('name', '=', $name)->first();
    $path = $request->file('image')->getClientOriginalName();

    $habitat->zoo_id = $zoo->id;
    $habitat->name = $request->name;
    $habitat->description = $request->description;
    $habitat->image = $path;

    if (!Storage::disk('public')->exists('asset/images')) {
        Storage::disk('public')->makeDirectory('asset/images');
    }
    Storage::disk('public')->putFileAs("asset/images", $request->file('image'), $path);

    $habitat->save();
    return redirect()->route('habitat', ['name' => $habitat->name]);
}
```

Pic.30

```php
public function delete(string $name): RedirectResponse
{
    $habitat = Habitat::query()->where('name', '=', $name)->first();
    Storage::disk('public')->delete('asset/images/' . $habitat->image);
    $habitat->delete();

    return redirect()->route('home');
}
```

Extraits de code : Sécurité
Policy, Gate

Pic.31

```php
<?php

namespace App\Policies;

use App\Models\User;
use Illuminate\Support\Facades\Gate;

class ServicesPolicy extends Gate
{
    public function create(User $user): bool
    {
        return $user->role === 'admin';
    }

    public function update(User $user): bool
    {
        return in_array($user->role, ['admin', 'employee']);
    }

    public function view(User $user): bool
    {
        return in_array($user->role, ['admin', 'employee']);
    }

    public function delete(User $user): bool
    {
        return $user->role === 'admin';
    }
}
```

Pic.32

```php
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Models\Service;
use App\Models\Zoo;
use App\Requests\ServicesFormRequest;
use Exception;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Gate;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;

class ServicesAdminController extends Controller
{
    public function index(): View
    {
        return view('page.services');
    }

    /**
     * @throws Exception
     */
    public function show(string $name): View|RedirectResponse
    {
        if (Gate::denies('view', Service::class)) {
            return redirect()->route('dashboard');
        }

        $service = Service::query()->where('name', '=', $name)->first();
        if (!$service) {
            throw new Exception("Il n'y a pas de service", 404);
        }
        return view('page.service', compact('service'));
    }
```

Pic.33

```php
public function update(ServicesFormRequest $request, string $name): RedirectResponse
    {
        if(Gate::denies('update', Service::class)) {
            return redirect()->route('dashboard');
        }

        $zoo = Zoo::query()->where('name', '=', 'Arcadia')->first();
        $service = Service::query()->where('name', '=', $name)->first();
        $path = $request->file('image')->getClientOriginalName();

        $service->zoo_id = $zoo->id;
        $service->name = $request->name;
        $service->description = $request->description;
        $service->image = $path;
        if (!Storage::disk('public')->exists($service->image)) {
            Storage::disk('public')->makeDirectory("assets/images");
        }
        Storage::disk('public')->putFileAs("asset/images",$request->file('image'), $path);

        $service->save();
        return redirect()->route('services.create');
    }
```

Extrait de code : **Base de données SQL**

1. CREATE & DROP DATABASE / CREATE TABLE

Pic.34

```sql
-- Supprimer la DB si elle existe
DROP DATABASE IF EXISTS `i-jurain_arcadia`;

-- Création de la DB avec des données d'exemples (Seeders)
CREATE DATABASE `i-jurain_arcadia`;

-- Utilisation de la base de données
USE `i-jurain_arcadia`;

-- Création des tables
CREATE TABLE `zoo`
(
    `id`          INT AUTO_INCREMENT,
    `name`        VARCHAR(255) NOT NULL,
    `description` TEXT,
    `created_at`  DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`  DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    PRIMARY KEY (id)
);

CREATE TABLE `users`
(
    `id`                INT PRIMARY KEY AUTO_INCREMENT,
    `zoo_id`            INT          NOT NULL,
    `name`              VARCHAR(255) NOT NULL,
    `email`             VARCHAR(255) NOT NULL UNIQUE,
    `email_verified_at` DATETIME     NULL,
    `password`          VARCHAR(255) NOT NULL,
    `role`              VARCHAR(255) NOT NULL,
    `created_at`        DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`        DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`zoo_id`) REFERENCES `zoo` (`id`)
);

CREATE TABLE `habitats`
(
    `id`          INT PRIMARY KEY AUTO_INCREMENT,
    `zoo_id`      INT          NOT NULL,
    `name`        VARCHAR(255) NOT NULL,
    `description` TEXT,
    `image`       VARCHAR(255),
    `created_at`  DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`  DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`zoo_id`) REFERENCES `zoo` (`id`)
);

CREATE TABLE `services`
(
    `id`          INT PRIMARY KEY AUTO_INCREMENT,
    `zoo_id`      INT          NOT NULL,
    `name`        VARCHAR(255) NOT NULL,
    `description` TEXT,
    `image`       VARCHAR(255),
    `created_at`  DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`  DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`zoo_id`) REFERENCES `zoo` (`id`)
);

CREATE TABLE `reviews`
(
    `id`         INT PRIMARY KEY AUTO_INCREMENT,
    `zoo_id`     INT  NOT NULL,
    `content`    TEXT NOT NULL,
    `status`     ENUM ('validated', 'rejected', 'pending') DEFAULT
'pending',
    `author`     VARCHAR(255),
    `rating`     INT CHECK (`rating` BETWEEN 1 AND 5),
    `created_at` DATETIME                                  DEFAULT
CURRENT_TIMESTAMP,
    `updated_at` DATETIME                                  DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`zoo_id`) REFERENCES `zoo` (`id`)
);
```

Pic.35

```sql
CREATE TABLE `openings`
(
    `id`         INT PRIMARY KEY AUTO_INCREMENT,
    `zoo_id`     INT          NOT NULL,
    `day_open`   VARCHAR(255) NOT NULL,
    `hour_open`  TIME         NOT NULL,
    `hour_close` TIME         NOT NULL,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`zoo_id`) REFERENCES `zoo` (`id`)
);

CREATE TABLE `animals`
(
    `id`          INT PRIMARY KEY AUTO_INCREMENT,
    `habitat_id`  INT          NOT NULL,
    `name`        VARCHAR(255) NOT NULL,
    `breed`       VARCHAR(255),
    `image`       VARCHAR(255),
    `description` TEXT,
    `created_at`  DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`  DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`habitat_id`) REFERENCES `habitats` (`id`)
);

CREATE TABLE `foods`
(
    `id`         INT PRIMARY KEY AUTO_INCREMENT,
    `name`       VARCHAR(255) NOT NULL,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

CREATE TABLE `foods_consum`
(
    `id`         INT PRIMARY KEY AUTO_INCREMENT,
    `animal_id`  INT         NOT NULL,
    `food_id`    INT         NOT NULL,
    `quantity`   INT         NOT NULL,
    `unit`       VARCHAR(10) NOT NULL,
    `user_id`    INT         NOT NULL,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`animal_id`) REFERENCES `animals` (`id`),
    FOREIGN KEY (`food_id`) REFERENCES `foods` (`id`),
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
);

CREATE TABLE `veterinarian_reports`
(
    `id`             INT PRIMARY KEY AUTO_INCREMENT,
    `animal_id`      INT NOT NULL,
    `animal_state`   VARCHAR(255),
    `food_consum_id` INT,
    `content`        TEXT,
    `user_id`        INT NOT NULL,
    `created_at`     DATETIME DEFAULT CURRENT_TIMESTAMP,
    `updated_at`     DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (`animal_id`) REFERENCES `animals` (`id`),
    FOREIGN KEY (`food_consum_id`) REFERENCES `foods_consum` (`id`),
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
);
```

2. INSERT INTO DB

On insert les données telles que la table Zoo avant la table Users pour respecter les contraintes d'intégrité des clés secondaires.

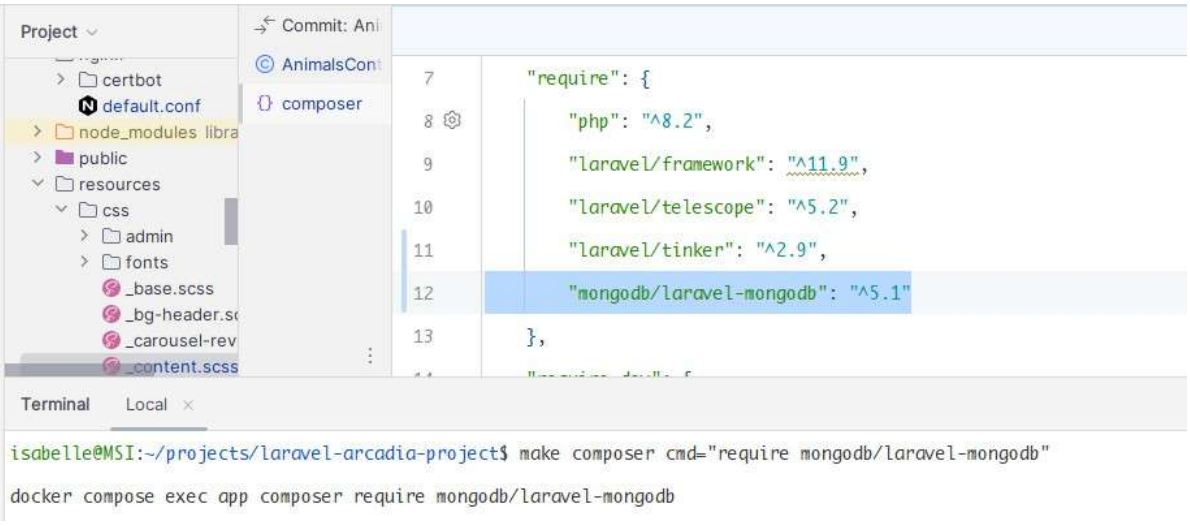
Pic.36

```
-- Insertion des données
INSERT INTO zoo (id, name, description)
VALUES (1, 'Arcadia',
        'Bienvenue à Arcadia, un zoo unique en son genre situé tout
près de la légendaire forêt de Brocéliande en Bretagne, en France.
Fondé en 1960, Arcadia vous invite à un voyage fascinant au cœur de la
faune japonaise. Découvrez une diversité exceptionnelle d\'animaux
originaires du Japon dans un cadre enchanteur et préservé.');

INSERT INTO users (id, zoo_id, name, email, password, role)
VALUES (1, 1, 'José Admin', 'joseadmin@gmail.com', '123456789aze',
'admin'),
       (2, 1, 'Josette Veterinarian', 'josetteveterinarian@gmail.com',
'123456789aze', 'veterinarian'),
       (3, 1, 'Joseph Employee', 'josephemployee@gmail.com',
'123456789aze', 'employee');
```

Extrait de code : **Base de données NoSQL**
Installation du package MongoDB pour Laravel (pic37)
Model, Controller et Migration MongoDB (pic38, 39a et 39b)

Pic.37



Pic.38

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use MongoDB\Laravel\Eloquent\Model
;

class ConsultAnimal extends Model
{
    use HasFactory;

    protected $table = 'consult_animals';
    protected $connection = 'mongodb';

    protected $fillable = [
        'consult_id',
        'animal_id',
        'view',
        'created_at',
        'updated_at',
    ];
}
```

Pic.39a

```php
public function show(string $name): View
    {
        $animal = Animal::query()->where('name', '=', $name)->with('habitat')->first();
        $consultAnimal = ConsultAnimal::query()->where('animal_id', '=', 1)->first();

        ++$consultAnimal->nb_view;

        $consultAnimal->save();

        if (!$animal) {
            throw new Exception("Il n'y a pas d'animal avec le nom $name", 404);
        }

        $animal->nb_view = $consultAnimal->nb_view;

        return view('page.animal-sheet', compact('animal'));
    }

    /**
     * @throws RuntimeException
     */
    public function getNumberOfViews(int $id): JsonResponse
    {
        $nbViews = ConsultAnimal::query()->where('animal_id', '=', 2)->first();

        if (!$nbViews) {
            throw new RuntimeException("Il n'y a pas d'animal avec l'id $id", 404);
        }

        return response()->json(compact('nbViews'));
    }
```

Pic.39b

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('consult_animals', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->integer('animal_id')->constrained()->cascadeOnDelete();
            $table->integer('nb_view')->default(0);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('consult_animals');
    }
};
```

Extraits de code : Docker

Dockerfile, default.conf, mongo-init, docker-compose et Makefile

Pic.40

```
# Étape 1 : Installation PHP et extensions
FROM php:8.3-fpm

RUN apt-get update \
    && apt-get install -y git \
    curl \

    unzip \
    libpng-dev \
    libjpeg-dev \
    libwebp-dev \
    libfreetype6-dev \
    libzip-dev \
    libonig-dev \
    libpq-dev \
    && docker-php-ext-configure gd --with-freetype --with-jpeg --with-webp \
    && docker-php-ext-install pdo_mysql mbstring zip gd \
    && pecl install mongodb \
    && docker-php-ext-enable mongodb

# Étape 2 : Installation Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
RUN composer -V

# Étape 3 : Installation Node.js et PNPM
RUN curl -fsSL https://deb.nodesource.com/setup_20.x | bash - \
    && apt-get install -y nodejs \
    && npm install -g npm pnpm \
    && pnpm config set store-dir /root/.pnpm-store  # Déplace le cache

# Définition du dossier de travail
WORKDIR /var/www/html

# Copie du projet
COPY . .

# Étape 4 : Installation et build avec PNPM
RUN pnpm install --frozen-lockfile \
    && pnpm build \
    && pnpm store prune  # Nettoie le cache

# Exposition du port
EXPOSE 8000

# Démarrage de l'application
CMD ["pnpm", "start"]
```

Pic.41

```
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name                                    ;

    ssl_certificate /etc/letsencrypt/live/${SERVER_NAME}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/${SERVER_NAME}/privkey.pem;

    root /var/www/html/public;
    index index.php;

    if ($host =                    ) {
      return 301 https://www.$host$request_uri;
    }

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass app:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}
```

Pic.42

```
db = db.getSiblingDb("zooarcadia");
db.createUser({
    user: "zooarcadia",
    pwd: "zooarcadia",
    roles: [
        {
            role: "readWrite",
            db: "zooarcadia"
        }
    ]
})
```

Pic.43

```yaml
services:
    app:
        build:
            context: .
            dockerfile: Dockerfile
        container_name: php-app
        working_dir: /var/www/html
        volumes:
            - ./:/var/www/html
        networks:
            - laravel_network
        command: php-fpm

    nginx:
        image: nginx:alpine
        container_name: nginx_server
        volumes:
            - ./:/var/www/html
            - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
            - ./nginx/certbot/conf:/etc/letsencrypt
            - ./nginx/certbot/www:/var/www/certbot
        ports:
            - "${APP_PORT}:80"
            - "443:443"
        depends_on:
            - app
        networks:
            - laravel_network
        environment:
            - SERVER_NAME=${APP_DOMAIN}
            - PORT=${APP_PORT}

    certbot:
        image: certbot/certbot:v3.1.0
        container_name: certbot
        volumes:
            - ./nginx/certbot/conf:/etc/letsencrypt
            - ./nginx/certbot/www:/var/www/certbot
            - ./:/var/www/html
        networks:
            - laravel_network
        depends_on:
            - nginx

    mongo:
        image: mongo:8
        container_name: mongo_db
        ports:
            - "27018:27017"
        environment:
            MONGO_INITDB_ROOT_USERNAME: ${MONGO_USERNAME}
            MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
        volumes:
            - mongo_data:/data/db
            - ./mongo-init.js:/docker-entrypoint-initdb.d/mongo-init.js
        networks:
            - laravel_network

networks:
    laravel_network:
        driver: bridge

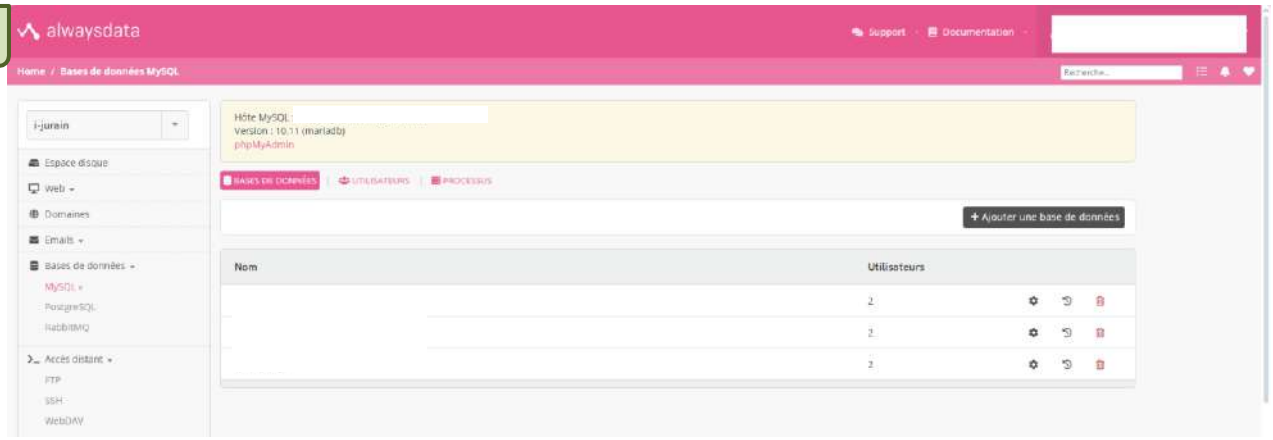
volumes:
    mongo_data:
```

Configuration de l'environnement de travail

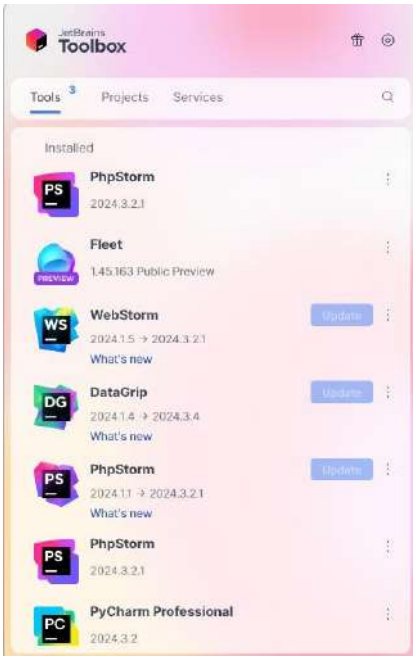
Pic.44



Pic.45

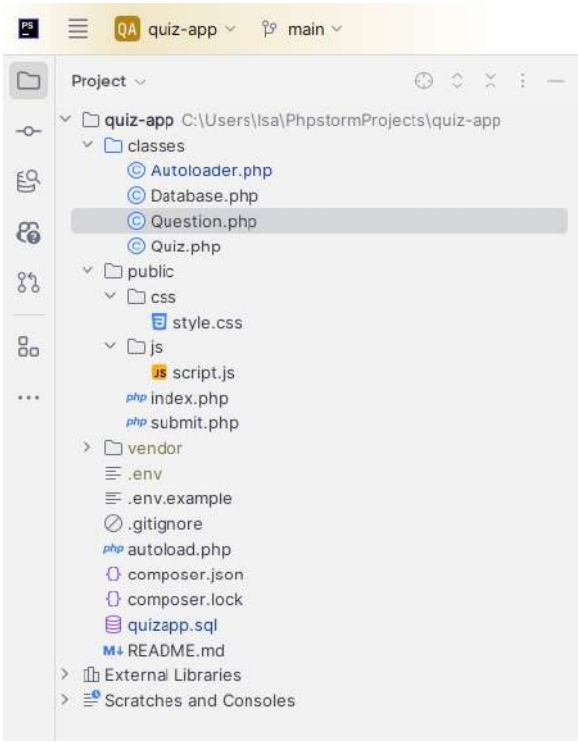


Pic.46



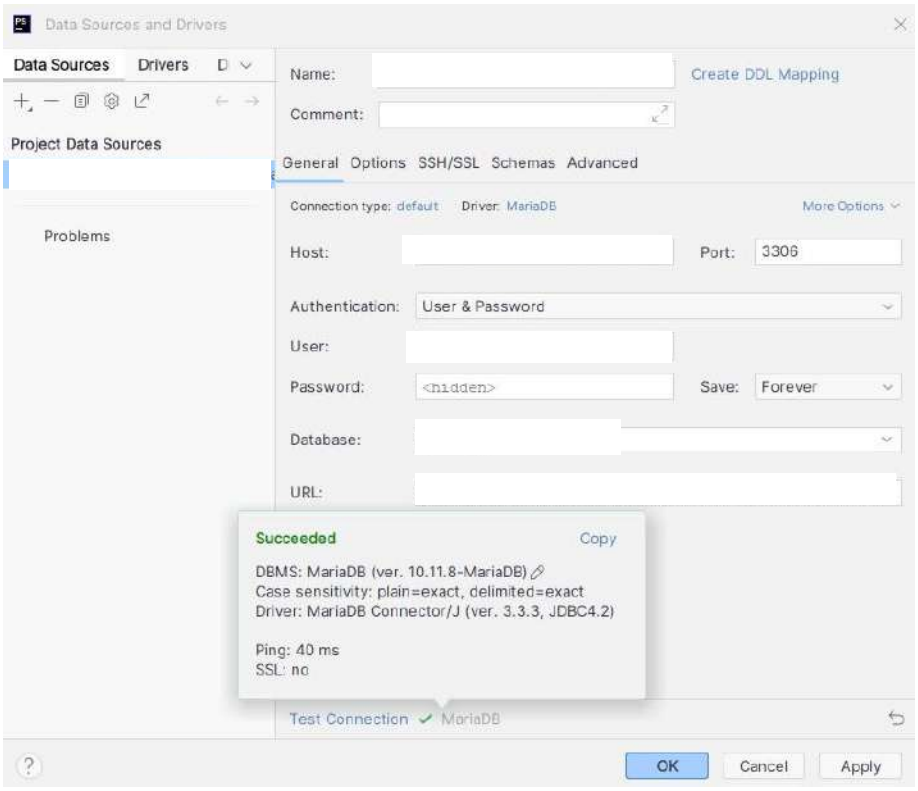
Application QuizApp

Pic.47



Structure du projet

Pic.48



Connexion DB

Création des tables *questions* et *answers*

Pic.49

```
CREATE TABLE IF NOT EXISTS questions
(
    id       CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    question TEXT NOT NULL,
    unique key (id)
);

-- Création de la table des réponses
CREATE TABLE IF NOT EXISTS answers
(
    id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    question_id CHAR(36) NOT NULL,
    answer      TEXT     NOT NULL,
    is_correct  BOOLEAN              DEFAULT FALSE,
    FOREIGN KEY (question_id) REFERENCES questions (id) ON DELETE CASCADE
);
```

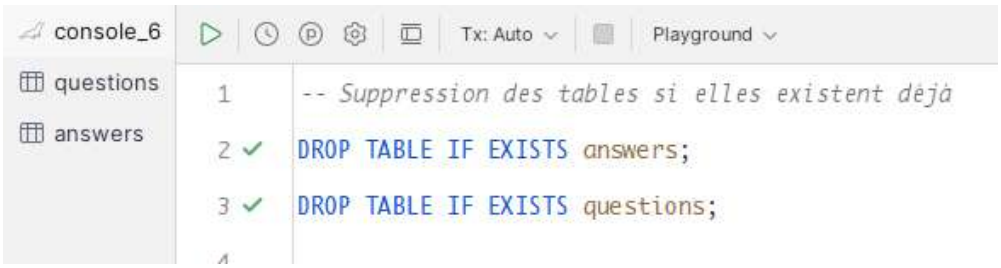
Requête effectuée :

Pic.50

```
i-jurain_quiz> CREATE TABLE IF NOT EXISTS questions
               (
                   id       CHAR(36) PRIMARY KEY DEFAULT (UUID()),
                   question TEXT NOT NULL,
                   unique key (id)
               )
[2025-03-10 12:28:09] completed in 24 ms
i-jurain_quiz> CREATE TABLE IF NOT EXISTS answers
               (
                   id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
                   question_id CHAR(36) NOT NULL,
                   answer      TEXT     NOT NULL,
                   is_correct  BOOLEAN              DEFAULT FALSE,
                   FOREIGN KEY (question_id) REFERENCES questions (id) ON DELETE CASCADE
               )
[2025-03-10 12:28:09] completed in 21 ms
```

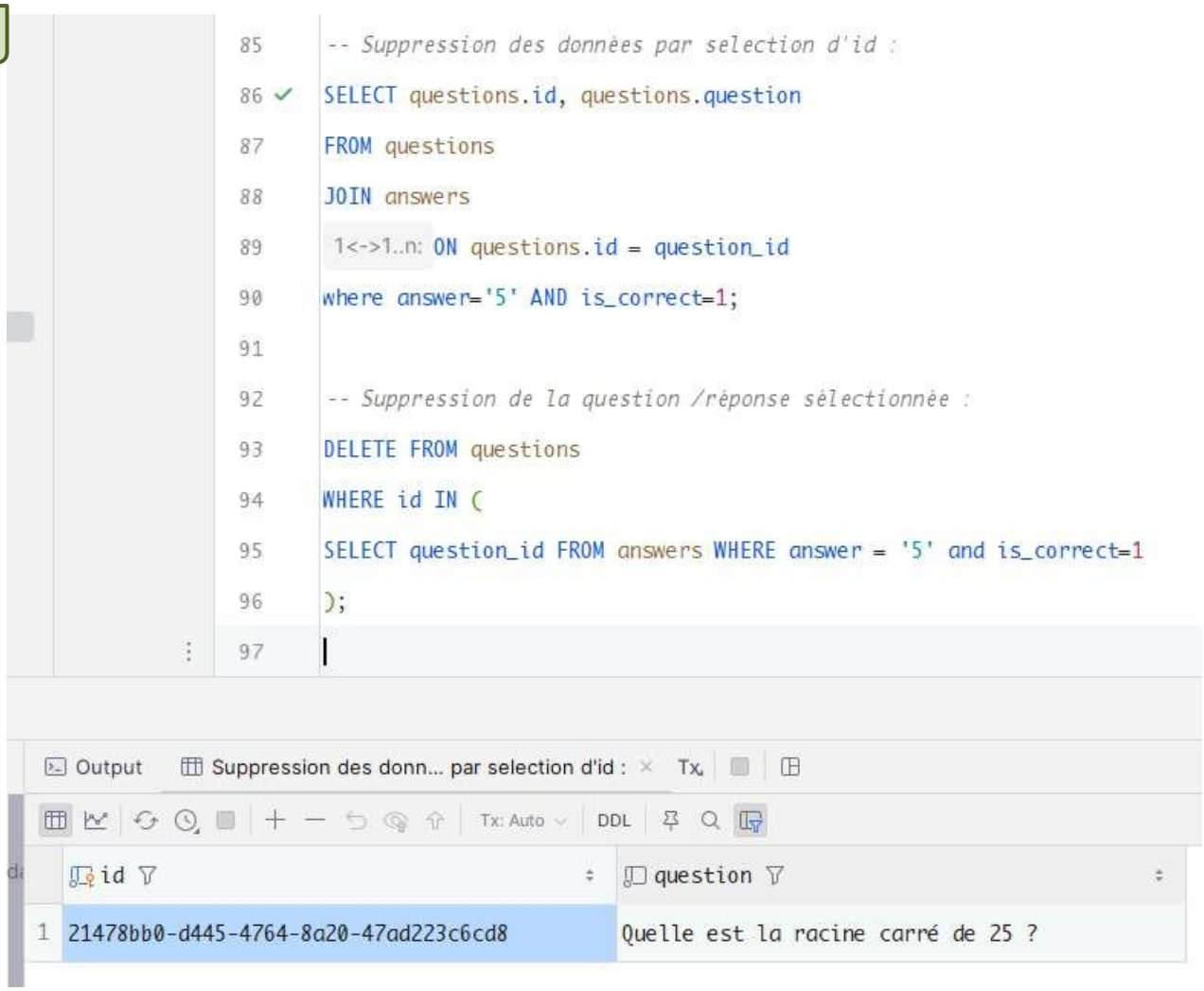
Requête pour supprimer si la table existe :

Pic.51



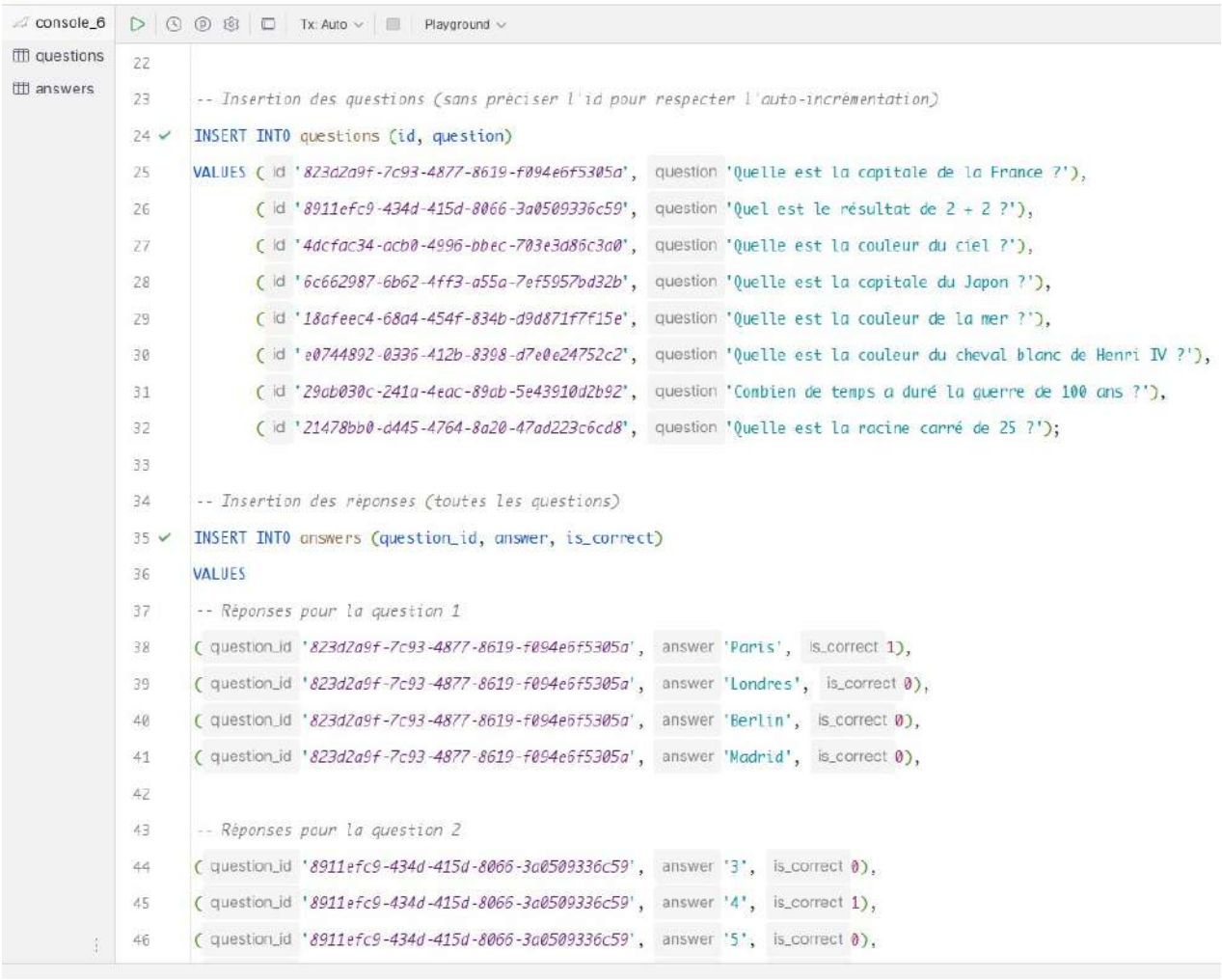
Exemple de requête avec jointure & requête de suppression :

Pic.52



Requêtes INSERT INTO dans la console :

Pic.53



UPDATE ANSWERS :

Pic.54

```
97
98  -- Update
99  UPDATE answers SET is_correct=0 WHERE question_id='e0744892-0336-412b-8398-d7e0e24752c2' AND answer='Blanche';
100 UPDATE answers SET is_correct=1 WHERE question_id='e0744892-0336-412b-8398-d7e0e24752c2' AND answer='Grise';
```

Sript SQL final :

Pic.55

```sql
-- Suppression des tables si elles existent déjà
DROP TABLE IF EXISTS answers;
DROP TABLE IF EXISTS questions;

-- Création de la table des questions
CREATE TABLE IF NOT EXISTS questions
(
    id       CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    question TEXT NOT NULL,
    unique key (id)
);

-- Création de la table des réponses
CREATE TABLE IF NOT EXISTS answers
(
    id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
    question_id CHAR(36) NOT NULL,
    answer      TEXT     NOT NULL,
    is_correct  BOOLEAN              DEFAULT FALSE,
    FOREIGN KEY (question_id) REFERENCES questions (id) ON DELETE CASCADE
);

-- Insertion des questions (sans préciser l'id pour respecter l'auto-incrémentation)
INSERT INTO questions (id, question)
VALUES ('823d2a9f-7c93-4877-8619-f094e6f5305a', 'Quelle est la capitale de la France ?'),
       ('8911efc9-434d-415d-8066-3a0509336c59', 'Quel est le résultat de 2 + 2 ?'),
       ('4dcfac34-acb0-4996-bbec-703e3d86c3a0', 'Quelle est la couleur du ciel ?'),
       ('6c662987-6b62-4ff3-a55a-7ef5957bd32b', 'Quelle est la capitale du Japon ?'),
       ('18afeec4-68a4-454f-834b-d9d871f7f15e', 'Quelle est la couleur de la mer ?'),
       ('e0744892-0336-412b-8398-d7e0e24752c2', 'Quelle est la couleur du cheval blanc de Henri IV ?'),
       ('29ab030c-241a-4eac-89ab-5e43910d2b92', 'Combien de temps a duré la guerre de 100 ans ?'),
       ('21478bb0-d445-4764-8a20-47ad223c6cd8', 'Quelle est la racine carré de 25 ?');

-- Insertion des réponses (toutes les questions)
INSERT INTO answers (question_id, answer, is_correct)
VALUES
-- Réponses pour la question 1
('823d2a9f-7c93-4877-8619-f094e6f5305a', 'Paris', 1),
('823d2a9f-7c93-4877-8619-f094e6f5305a', 'Londres', 0),
('823d2a9f-7c93-4877-8619-f094e6f5305a', 'Berlin', 0),
('823d2a9f-7c93-4877-8619-f094e6f5305a', 'Madrid', 0),

-- Réponses pour la question 2
('8911efc9-434d-415d-8066-3a0509336c59', '3', 0),
('8911efc9-434d-415d-8066-3a0509336c59', '4', 1),
('8911efc9-434d-415d-8066-3a0509336c59', '5', 0),
('8911efc9-434d-415d-8066-3a0509336c59', '6', 0),

-- Réponses pour la question 3
('4dcfac34-acb0-4996-bbec-703e3d86c3a0', 'Bleu', 1),
('4dcfac34-acb0-4996-bbec-703e3d86c3a0', 'Vert', 0),
('4dcfac34-acb0-4996-bbec-703e3d86c3a0', 'Rouge', 0),
('4dcfac34-acb0-4996-bbec-703e3d86c3a0', 'Jaune', 0),

-- Réponses pour la question 4
('6c662987-6b62-4ff3-a55a-7ef5957bd32b', 'Tokyo', 1),
('6c662987-6b62-4ff3-a55a-7ef5957bd32b', 'Pékin', 0),
('6c662987-6b62-4ff3-a55a-7ef5957bd32b', 'Séoul', 0),
('6c662987-6b62-4ff3-a55a-7ef5957bd32b', 'Bangkok', 0),

-- Réponses pour la question 5
('18afeec4-68a4-454f-834b-d9d871f7f15e', 'Bleue', 1),
('18afeec4-68a4-454f-834b-d9d871f7f15e', 'Verte', 0),
('18afeec4-68a4-454f-834b-d9d871f7f15e', 'Rouge', 0),
('18afeec4-68a4-454f-834b-d9d871f7f15e', 'Jaune', 0),

-- Réponses pour la question 6
('e0744892-0336-412b-8398-d7e0e24752c2', 'Blanche', 1),
('e0744892-0336-412b-8398-d7e0e24752c2', 'Noire', 0),
('e0744892-0336-412b-8398-d7e0e24752c2', 'Grise', 0),
('e0744892-0336-412b-8398-d7e0e24752c2', 'Marron', 0),

-- Réponses pour la question 7
('29ab030c-241a-4eac-89ab-5e43910d2b92', '116 ans', 1),
('29ab030c-241a-4eac-89ab-5e43910d2b92', '100 ans', 0),
('29ab030c-241a-4eac-89ab-5e43910d2b92', '50 ans', 0),
('29ab030c-241a-4eac-89ab-5e43910d2b92', '200 ans', 0),

-- Réponses pour la question 8
('21478bb0-d445-4764-8a20-47ad223c6cd8', '5', 1),
('21478bb0-d445-4764-8a20-47ad223c6cd8', '2', 0),
('21478bb0-d445-4764-8a20-47ad223c6cd8', '10', 0),
('21478bb0-d445-4764-8a20-47ad223c6cd8', '4', 0);

-- Suppression des données par selection d'id :
# SELECT questions.id, questions.question
# FROM questions
# JOIN answers
# ON questions.id = question_id
# where answer='5' AND is_correct=1;
#
# -- Suppression de la question /réponse sélectionnée :
# DELETE FROM questions
# WHERE id IN (
# SELECT question_id FROM answers WHERE answer = '5' and is_correct=1
# );

# -- Update
# UPDATE answers SET is_correct=0 WHERE question_id='e0744892-0336-412b-8398-d7e0e24752c2' AND answer='Blanche';
# UPDATE answers SET is_correct=1 WHERE question_id='e0744892-0336-412b-8398-d7e0e24752c2' AND answer='Grise';
```

QuizApp !

Pic.56

